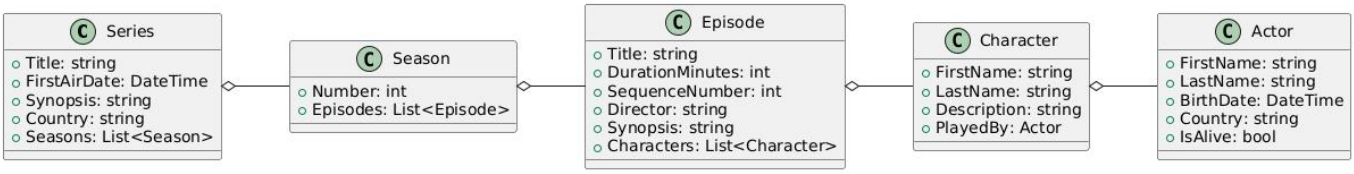
Sérialisation

> **Fallacies :** 3 (bande passante infinie), 8 (réseau homogène).
> **À retenir :** sérialiser = transformer un objet en mémoire en octets que **d'autres programmes, dans d'autres langages**, pourront relire. Le format est un **contrat** : le modifier sans prévenir casse les autres.

La sérialisation (*marshalling*) convertit des données ou des objets en une séquence de bits, pour les **transmettre** ou les **stocker**. Le format peut être du texte (JSON, XML) ou du binaire. L'opération inverse est la **désérialisation** (*unmarshalling*).

Un exemple

Un modèle de données de séries TV :



Un acteur existe en mémoire dans notre programme, sous forme d'objet. Les autres parties du programme y accèdent car elles partagent la même mémoire. Pour retrouver ce même acteur dans **un autre programme** (même machine ou ailleurs), on le **sérialise** en JSON :

```json
{
  "FirstName": "Jennifer",
  "LastName": "Aniston",
  "BirthDate": "1969-02-11T00:00:00",
  "Country": "USA",
  "IsAlive": true
}
```

Ce texte se stocke dans un fichier lisible et éditable, ou s'envoie par le réseau (HTTP, MQTT, TCP/IP). Le récepteur le **désérialise** pour recréer l'objet dans *sa* mémoire, quel que soit son langage.

Pourquoi utiliser la sérialisation ?

Dans les systèmes distribués, les composants sont répartis sur plusieurs machines et ne partagent aucune mémoire : il faut transférer des données structurées sans perdre d'information (une commande d'achat avec client et produits, un état de maison, un message de chat). La sérialisation sert aussi à **sauvegarder l'état** d'un objet pour le restaurer plus tard, et à l'**interopérabilité** entre systèmes hétérogènes.

Quels sont les formats les plus connus ?

- **Texte** : JSON (simple, léger, omniprésent dans les API REST) et XML (extensible, plus lourd).
- **Binaire** : Protocol Buffers (Google), Avro (Apache), MessagePack… plus compacts et plus rapides à lire et écrire, utiles quand la performance compte.

Binaire VS Texte

	Texte (JSON, XML)	**Binaire (Protobuf, MessagePack…)**
Lisibilité, débogage	**lisible par un humain**	illisible sans outil ni schéma
Taille	plus volumineux (noms de champs répétés, base64 pour les images)	plus compact
Vitesse	plus lent à produire et à lire	plus rapide
Interopérabilité	immédiate, aucun outil particulier	exige de partager un schéma

Le texte a surtout l'avantage d'être **lisible**. Le binaire compact **n'est pas auto-descriptif** : sans le schéma, on ne sait pas le lire. Mesurez : D01.

Comment fonctionne la sérialisation ?

Elle « parcourt » l'objet, en extrait les attributs et les traduit dans un format conforme au schéma choisi. La désérialisation relit le flux pour recréer l'objet.

Défis et limites de la sérialisation

- **Dates et heures.** Les formats varient d'une culture à l'autre : mettez-vous d'accord au sein du système. Le standard mondial est ISO 8601 (en **UTC** pour les échanges).
- **Évolution et versions.** Si la structure change entre la sérialisation et la désérialisation, l'objet ne se reconstruit plus. Parades : **versionner** le format, ne **jamais renommer** un champ sans prévenir, ajouter des champs **optionnels**, être *strict en émission et tolérant en réception*.
- **Sécurité.** Désérialiser des données externes peut ouvrir des failles (injection, instanciation de types arbitraires) : **validez** les données reçues contre un schéma avant de les utiliser.
- **Performance.** Objets complexes ou gros volumes : le texte génère plus de données que le binaire. Une image encodée en base64 grossit d'un tiers.
- **Constructeurs et reconstruction.** Le désérialiseur doit savoir *comment* recréer l'objet : constructeur sans paramètre puis assignation, ou constructeur dont les paramètres correspondent aux champs. S'il y a plusieurs constructeurs, il faut lui désigner le bon.
- **Champs sensibles.** Mot de passe, jeton, clé d'API ne doivent pas être sérialisés : **excluez-les explicitement** (le champ n'apparaît alors ni dans l'écriture ni dans la lecture).

Chaque bibliothèque a son mécanisme pour les deux derniers points ; le principe est le même :

Langage / bibliothèque	**Désigner le constructeur**	**Exclure un champ**
C# `System.Text.Json`	`[JsonConstructor]`	`[JsonIgnore]`
Java Jackson	`@JsonCreator`	`@JsonIgnore`
JavaScript	fonction de reconstruction (`reviver`)	`toJSON()` ou `replacer`
Python	`dataclasses` / `pydantic`	`pydantic` : `exclude`

Pour s'exercer

- Drill : D01 · Chat : C1 · Contrat : contrat de base
- Activité alternative : Sérialise ma série